

ME315 - Lab 02

Pedro Rocha Teixeira - 253670

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(ggcal)
library(ggplot2)
```

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada (`ARRIVAL_DELAY > 10`)?

Resposta: A soma de voos atrasados e o total de voos válidos, dividindo a soma de voos atrasados pelo total de voos válidos obtemos uma boa proporção real.

2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):
 - a. Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;
 - b. Remova observações que tenham valores faltantes em campos de interesse;
 - c. Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
 - d. Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe `tibble`;

- e. A função deve receber apenas dois argumentos:
- f. input: o conjunto de dados (referente ao lote em questão);
- ii. pos: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
getStats <- function(input, pos) {  
  input %>%  
    # a.  
    filter(AIRLINE %in% c("AA", "DL", "UA", "US")) %>%  
    # b.  
    filter(!is.na(YEAR), !is.na(MONTH), !is.na(DAY),  
           !is.na(AIRLINE), !is.na(ARRIVAL_DELAY)) %>%  
    # c.  
    group_by(YEAR, MONTH, DAY, AIRLINE) %>%  
    # d.  
    summarise(  
      voos_atrasados = sum(ARRIVAL_DELAY > 10),  
      total_voos      = n(),  
      .groups = "drop"  
    )  
}
```

- 3. Utilize alguma função readr::read_***_chunked para importar o arquivo flights.csv.zip.
 - a. Configure o tamanho do lote (chunk) para 100 mil registros;
 - b. Configure a função de callback para instanciar DataFrames aplicando a função getStats criada acima;
 - c. Configure o argumento col_types de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```
tipos_colunas <- cols_only(  
  YEAR = col_integer(),  
  MONTH = col_integer(),  
  DAY = col_integer(),  
  AIRLINE = col_character(),  
  ARRIVAL_DELAY = col_double()  
)  
  
estatisticas_acumuladas <- read_csv_chunked(  
  file = "flights.csv",  
  callback = DataFrameCallback$new(getStats),
```

```

chunk_size = 100000,
col_types = tipos_colunas
)

```

4. Crie uma função chamada `computeStats` que:

- a. Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
- b. Retorne as informações em um tibble contendo apenas as seguintes colunas:
- c. Cia: sigla da companhia aérea;
- ii. Data: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`);
- iii. Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo [0,1].

```

computeStats <- function(df_stats) {
  df_stats %>%
    # a.
    group_by(YEAR, MONTH, DAY, AIRLINE) %>%
    summarise(
      total_atrasados = sum(voos_atrasados),
      total_voos      = sum(total_voos),
      .groups         = "drop"
    ) %>%
    # b.
    transmute(
      Cia = AIRLINE,
      Data = as.Date(paste(YEAR, MONTH, DAY, sep = "-")),
      Perc = total_atrasados / total_voos
    )
}

stats_finais <- computeStats(estatisticas_acumuladas)

```

5. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

- a. Instale e carregue os pacotes `ggcal` e `ggplot2`.
- b. Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.
- c. Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá:

- d. Criar um subconjunto de stats de forma a conter informações de atraso e data apenas da Cia. Aérea dada por cia.
- ii. Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, x representa as datas de interesse e y, os percentuais de atraso para as datas descritas em x.
- iii. Retornar para o usuário a base do calendário criada acima.
- iv. Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`.
- e. Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Título")`.

```
# a.
pal <- scale_fill_gradient(
  low = "#4575b4",
  high = "#d73027",
  name = "Atrasos (%)",
  labels = scales::percent_format(accuracy = 1)
)

# c.
baseCalendario <- function(stats, cia) {
  # i.
  dados_cia <- stats %>%
    filter(Cia == cia)

  # ii.
  g <- ggcal(dados_cia$Data, dados_cia$Perc)

  # iii.
  return(g)
}

# d.
cAA <- baseCalendario(stats_finais, "AA")
cDL <- baseCalendario(stats_finais, "DL")
cUA <- baseCalendario(stats_finais, "UA")
cUS <- baseCalendario(stats_finais, "US")

# e.
p_AA <- cAA + pal + ggtitle("Percentual de Atrasos na Chegada (>10 min) - American Airlines")
```

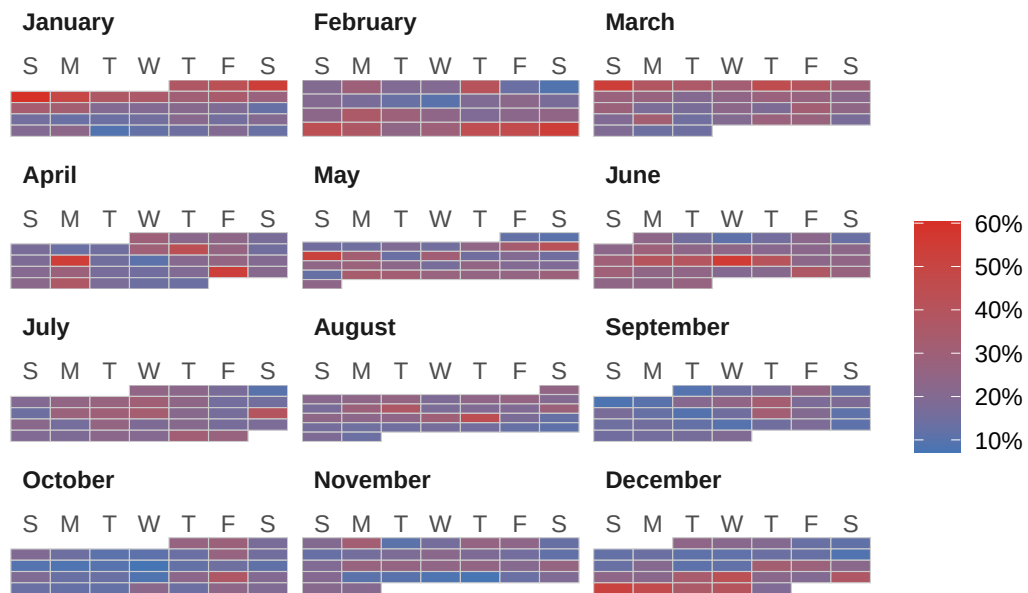
```

p_DL <- cDL + pal + ggtitle("Percentual de Atrasos na Chegada (>10 min) - Delta Air Lines (DL)")
p-UA <- cUA + pal + ggtitle("Percentual de Atrasos na Chegada (>10 min) - United Airlines (UA)")
p-US <- cUS + pal + ggtitle("Percentual de Atrasos na Chegada (>10 min) - US Airways (US)")

print(p_AA)

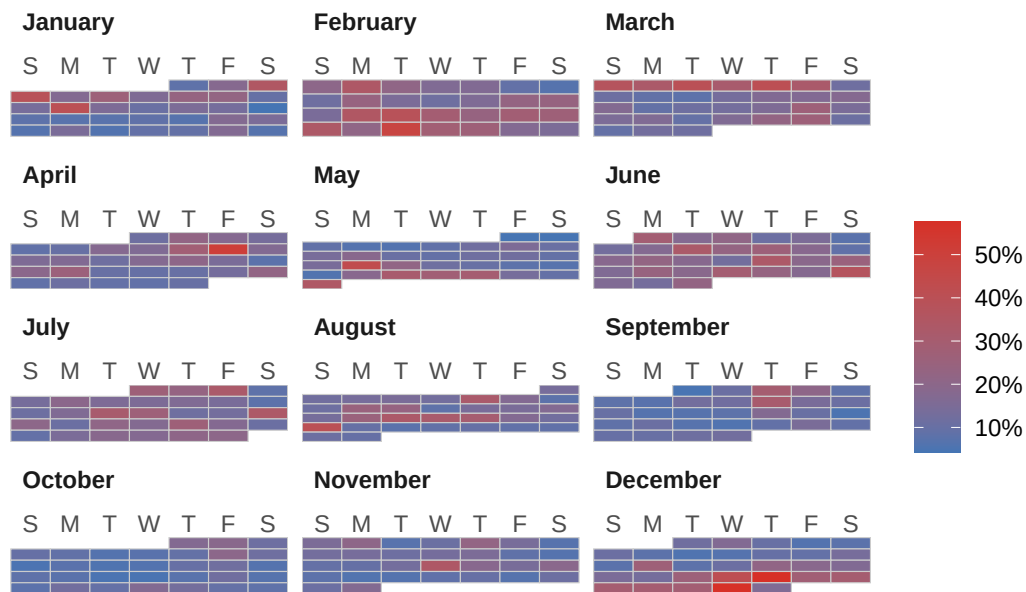
```

Percentual de Atrasos na Chegada (>10 min) – American Airlines (AA)



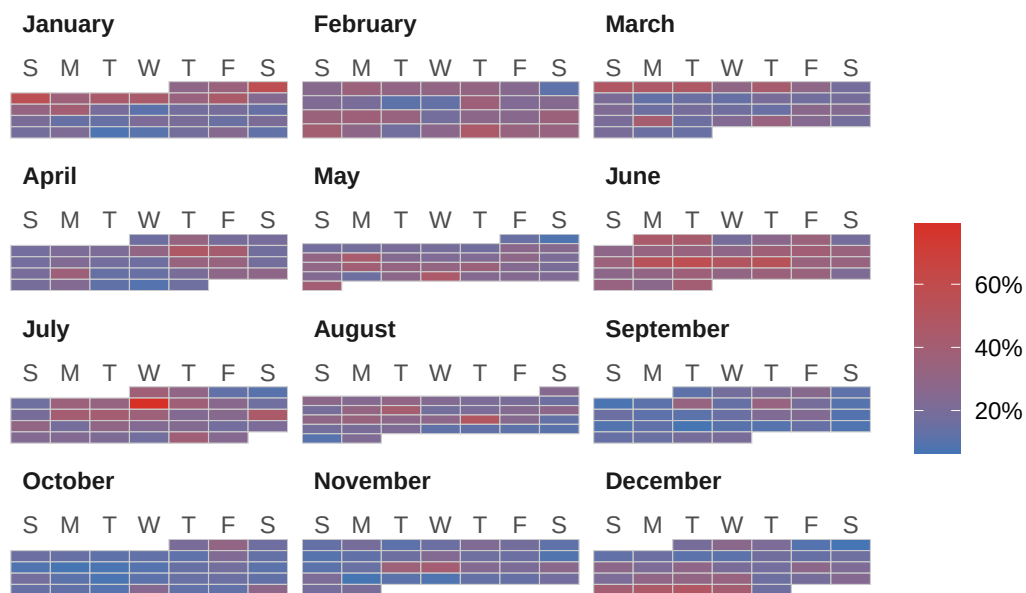
```
print(p_DL)
```

Percentual de Atrasos na Chegada (>10 min) – Delta Air Lines (DL)



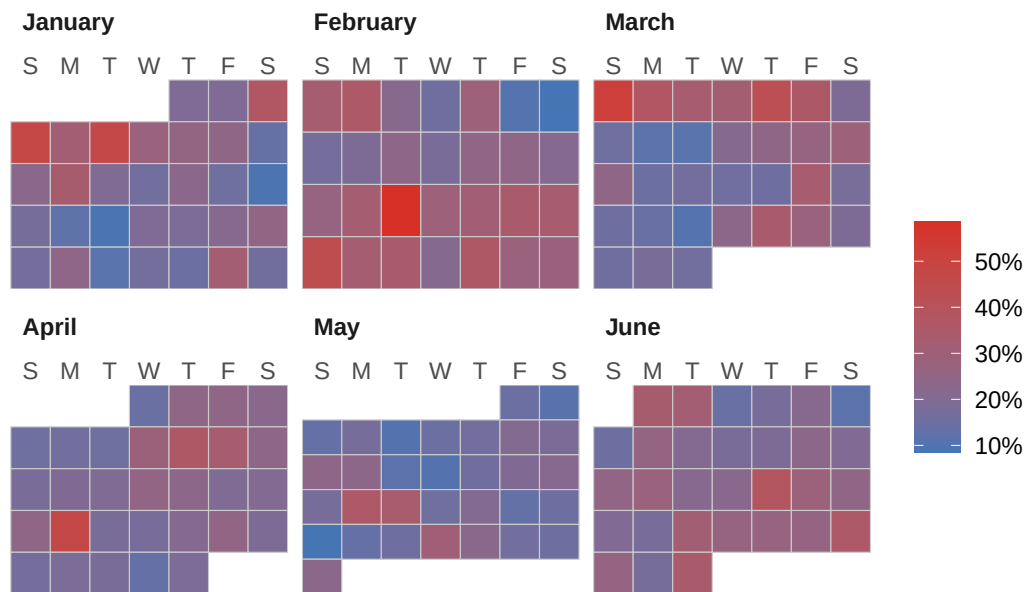
```
print(p_UA)
```

Percentual de Atrasos na Chegada (>10 min) – United Airlines (UA)



```
print(p_US)
```

Percentual de Atrasos na Chegada (>10 min) – US Airways (US)



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-23 16:12:07 -03"
```